

Manhwa → Anime

An open-source AI pipeline

Technical proposal: architectural problems, open-source model selection and a phase-by-phase plan for an end-to-end prototype.

PROTOTYPE SCOPE 2-3 pages → 15-40 second animated clip

Version 0.1 (draft for review)

6 October 2026

Model and license facts were gathered from web sources on 6 October 2026 and are marked ✓ (checked against a source) or † (general knowledge, verify on the model card before use).

Contents

Contents	2
1. Executive summary	3
Why hybrid rather than "animate everything with AI"	3
Default open-source stack	3
2. Scope, assumptions and constraints	3
3. Existing architectures	4
4. Proposed architecture	4
4.1 Orchestration and deployment view	5
4.2 Data contract: panel JSON	6
4.3 Pipeline state (LangGraph)	6
5. Architectural problems register	6
6. Open-source model selection	8
6.1 Perception and understanding	8
6.2 Cleanup and video generation	8
6.3 Audio and platform	9
6.4 Compute tiers	9
7. Shot routing and quality control	10
QC checks (all automatic, thresholds calibrated in Phase 3)	10
8. Phase-by-phase implementation plan	11
9. Evaluation plan and success criteria	12
10. Open questions and next steps	13
Questions that change the plan	13
Immediate next steps	13
Appendix A. Suggested repository layout	14
Appendix B. Sources consulted (6 October 2026)	14
Existing systems	14
Video models	14
Understanding, OCR and cleanup	14
Audio	14

1. Executive summary

Goal. Build an open-source-only prototype that turns 2–3 pages of a manhwa into a short animated, anime-style clip while preserving the story: panel order, dialogue, speakers, characters and mood.

Approach. A staged, resumable pipeline: **perceive** (panels, text, characters), **understand** (structured scene JSON from an open vision-language model), **generate** (image-to-video for the most important panels, 2.5D parallax or Ken Burns for the rest) and **compose** (voices, timeline, FFmpeg). Every panel is routed to the cheapest technique that still looks good, and every expensive step has a fallback, so a run always produces a watchable clip.

Why hybrid rather than "animate everything with AI"

Our market review found two things. First, motion-comic pipelines (panel detection plus pan, zoom and parallax) are reliable today, while pure image-to-video on stylized panels is where characters morph, line art bleeds and colors shift. Second, existing open repositories are small hobby projects: one covers comic-to-video without dialogue or audio, the other covers story-to-anime with paid APIs. The proposal combines the useful parts of each and swaps every paid component for an open-weight one.

Default open-source stack

- **Perception:** Magiv3 (panels, text, characters, speech-bubble tails, OCR), comic-text-detector, OpenCV gutter slicing as the cheap first pass.
- **Understanding:** Qwen3-VL (8B default; 4B/2B for small GPUs) producing schema-constrained JSON per panel.
- **Cleanup:** LaMa inpainting to erase lettering before animation (video models garble text).
- **Video:** Wan 2.2 (Apache 2.0) as primary, with HunyuanVideo 1.5 and LTX-2.3 as bake-off candidates.
- **Voices:** Chatterbox (MIT) with Kokoro (Apache 2.0) as the lightweight fallback.
- **Orchestration:** LangGraph state machine, FastAPI + Redis job layer, ComfyUI for video workflows, FFmpeg for assembly.

Biggest risks: character and style consistency in generated video, text corruption inside generated frames, GPU memory, Korean/Japanese OCR and voices, and the fact that Magi was trained on manga (its behaviour on color, vertical-scroll manhwa is untested). Section 5 lists all sixteen architectural problems with mitigations.

Plan. Seven phases (P0–P6), about 22 working days for one developer (Section 8), with explicit exit criteria and an evaluation protocol (Section 9). The first usable artifact, a baseline motion-comic clip, arrives at the end of Phase 1.

2. Scope, assumptions and constraints

	Details
In scope	2–3 manhwa pages (vertical-scroll, typically full color) · output 9:16 MP4, 15–40 s · dialogue as TTS plus subtitles · open-weight models only · one machine or one rented GPU · CLI plus a minimal web UI · human checkpoints for fixing the pipeline's JSON.
Out of scope (v0.1)	Full chapters or episodes · lip-sync · per-character LoRA training (listed as a stretch) · music scoring · multi-language dubbing · production scaling · distribution or monetization.
Assumptions	You own or hold a license for the sample pages · source language is Korean, Japanese or English (the pipeline is language-parameterized) · "anime-like" means a 2D cel-shaded look with believable motion, not hand-drawn in-betweens · GPU access is unconfirmed, so three compute tiers are defined in Section 6.

3. Existing architectures

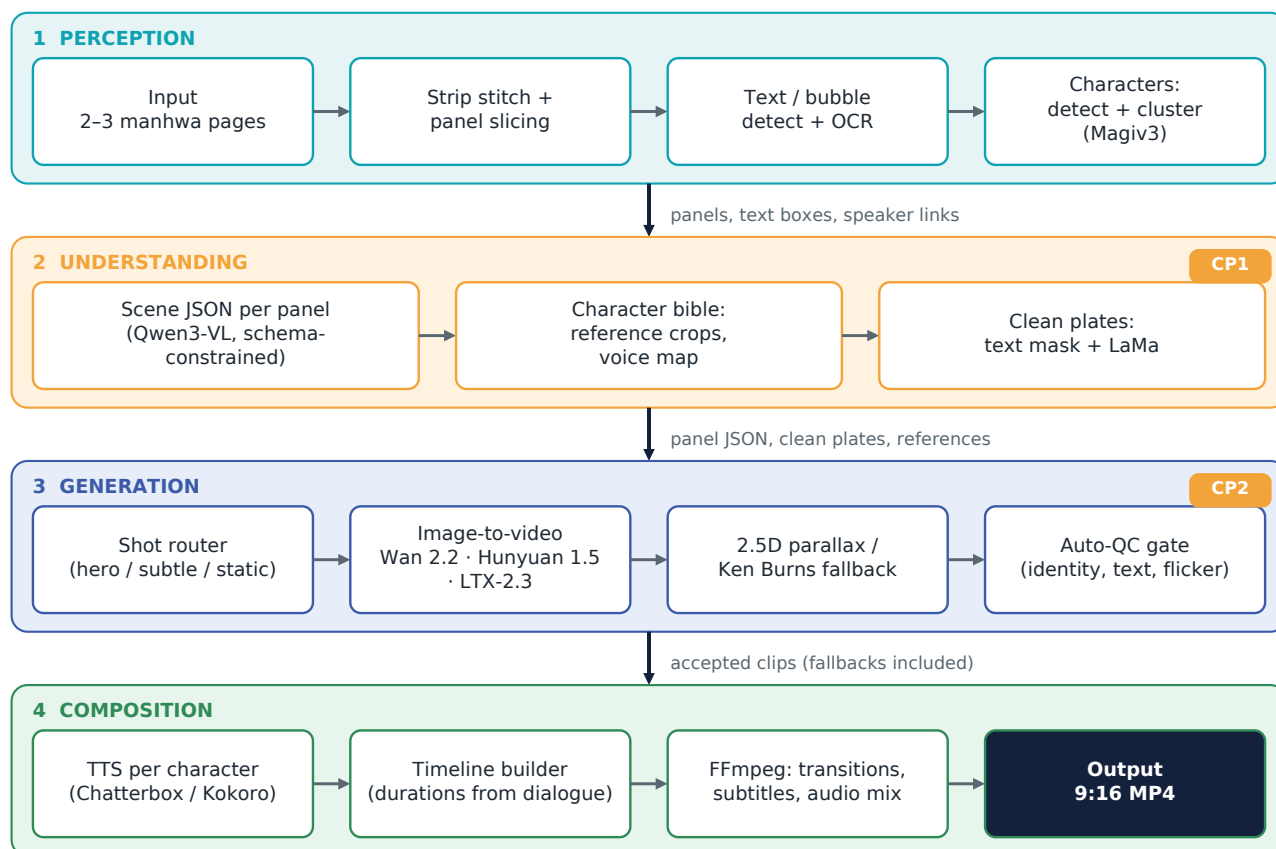
We inspected the two closest open projects, the main research model for comic understanding, and the hosted-tool category.

System	Pipeline	What to adopt	Gap for us
ACMP comic → motion comic (MIT, Python)	Ingest → panel detection (OpenCV or YOLOv8) → LLM scene analysis (Claude or Ollama) → animation (Wan VACE or Ken Burns) → transitions → FFmpeg. CLI, FastAPI and Streamlit entry points.	End-to-end skeleton; Ken Burns fallback; separate detector module; job API shape.	No dialogue or audio; detector benchmarked only on synthetic pages; tiny project (0 stars, 18 commits on 6 Oct 2026).
anime-pipeline story → anime (Python, asyncio)	Character proposal and lock (reference image + seed) → story → scene JSON → prompt builder with continuity anchors → image or hybrid shots → TTS → FFmpeg.	Checkpointed state, immutable pipeline state, character locking, budget-aware shot routing, cost tracker.	Starts from text, not existing art; relies on paid APIs (OpenAI, fal, Seedance, Runway, ElevenLabs); 1 star.
Magi / Magiv2 / Magiv3 research models	Detect panels, text and characters; cluster characters; link dialogue to speakers; v2 adds chapter-wide character names; v3 adds speech-bubble tails, OCR and character grounding.	Best available perception component, including speaker attribution.	Trained on manga, not verified on color manhwa or Korean text.
Hosted tools (Frameo and similar)	Upload panels, get animated video with effects and pacing.	Product ideas only (pacing, export formats).	Closed, not inspectable, not open-source.
Manual workflows (Luma / Kling on panels)	Upscale panel, write camera and motion prompt per panel, stitch results by hand.	Prompt patterns: camera moves plus "what should move".	Per-panel manual effort; closed models.

Resulting design choices. Use ACMP's skeleton and Ken Burns safety net; use anime-pipeline's checkpoints, state handling, character lock and budget router; use Magiv3 for perception; replace every paid API; never ask a video model to render a whole page at once.

4. Proposed architecture

Four layers, each producing a versioned artifact that the next layer consumes. Any stage can be re-run alone, which is what makes the human checkpoints and caching cheap.



CP = human checkpoint (fix JSON after Understanding; approve routing after the Router).

Diagram 1. End-to-end pipeline. Arrows between bands name the artifact handed over; amber tags mark human checkpoints.

4.1 Orchestration and deployment view

A thin job API accepts pages and configuration, a worker runs the LangGraph state machine, and heavy models sit behind small services so only one large model occupies the GPU at a time. Stages are idempotent and keyed by a content hash of their inputs, so retries and reruns reuse finished work.

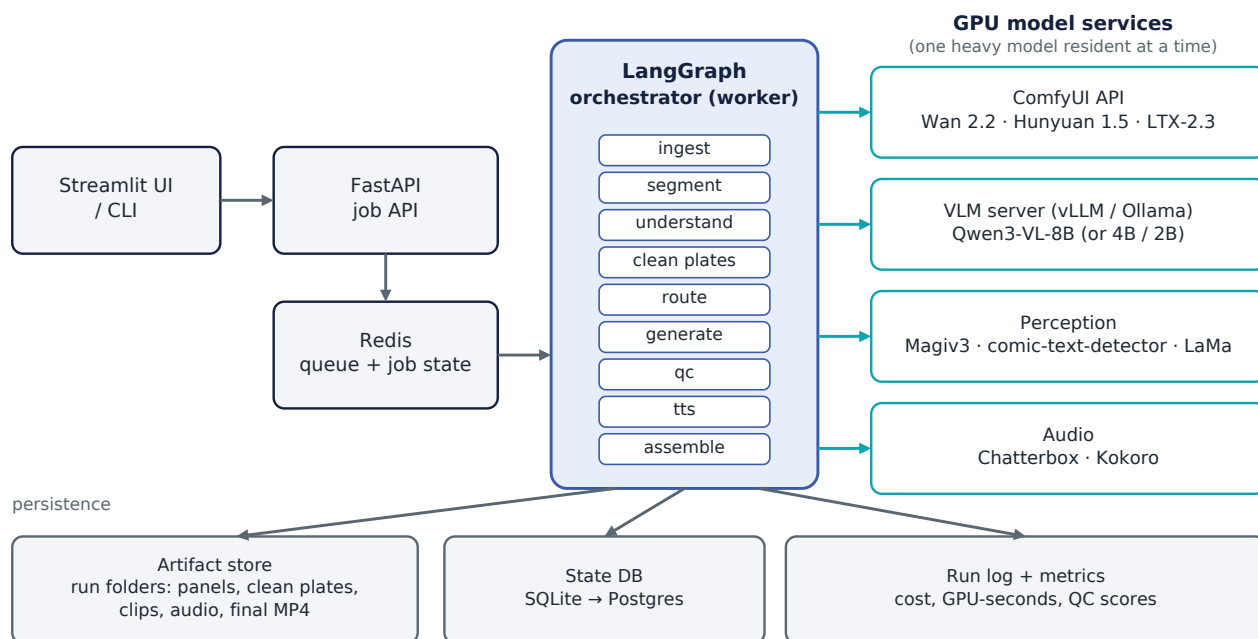


Diagram 2. Component view. The orchestrator talks to model services over local HTTP or file handoff; all outputs land in a per-run artifact folder.

4.2 Data contract: panel JSON

One JSON object per panel is the contract between understanding and generation. The VLM must fill it under a JSON schema; humans can edit it at checkpoint CP1.

```
{
  "panel_id": "p03",
  "bbox": [x, y, w, h],
  "shot": "close_up | medium | wide | splash",
  "characters": [{"id": "c1", "name": "Jin", "emotion": "shocked", "pose": "..."}],
  "action": "one-sentence description of what is happening",
  "dialogue": [{"speaker": "c1", "text": "...", "lang": "ko",
    "emotion": "urgent", "bbox": [x, y, w, h], "confidence": 0.93}],
  "sfx": [{"text": "...", "kind": "impact"}],
  "motion_prompt": "slow push-in, hair sways, dust drifts",
  "style_anchor": "2D anime, flat colors, clean line art",
  "motion_intensity": 0.3,
  "route": "i2v | parallax | kenburns",
  "duration_s": 4.0
}
```

4.3 Pipeline state (LangGraph)

State holds the run id, config hash, page and panel lists, panel JSON, character bible, clean-plate paths, clip candidates with QC scores, audio tracks and a cost ledger. Nodes: **ingest** → **segment** → **understand** → **[CP1]** → **clean_plates** → **route** → **[CP2]** → **generate** → **qc** → **(retry or fallback)** → **tts** → **assemble**. Checkpoints persist the state so a crash or a rejected checkpoint resumes from that node.

5. Architectural problems register

Sixteen problems that decide whether the prototype works. Risk is the author's judgement of impact times likelihood for this prototype, not a measured value.

ID	Problem	Why it is hard	Mitigation	Risk
P1	Panel segmentation on vertical strips	Borderless panels, overlapping art, full-bleed splashes, gutters that are not pure white.	Gutter slicing first; Magiv3 or a fine-tuned detector as fallback; hand-label 3 pages to measure F1; CP1 manual fix.	Medium
P2	Reading order and scene grouping	Scroll order is mostly top-to-bottom, but side-by-side and inset panels break simple sorting.	Sort by y then x with tolerance; VLM verifies sequence from thumbnails.	Low
P3	Text in art (bubbles, captions, SFX)	Korean, Japanese and English, vertical text and stylized SFX; Manga OCR is Japanese-only.	Language-routed OCR (Qwen3-VL OCR, Manga OCR for JA, PaddleOCR†); LLM cleanup; low-confidence lines flagged at CP1.	Medium
P4	Speaker attribution and identity	Speech-bubble tails must be linked to characters; one character looks different across poses and angles.	Magiv3 tails and character grounding, cross-checked by the VLM; character bible; CP1.	High
P5	Character consistency across clips	Image-to-video models reinterpret faces, hair and outfits; no memory across clips.	Always condition on the original panel pixels; short clips; identity-similarity QC; per-character LoRA as a stretch goal.	High
P6	Style drift under motion	Line work bleeds and flat colors turn photoreal; stylized panels are out-of-distribution for general video models.	Motion-only prompts with style anchors; low motion intensity; model bake-off; fallback lane.	High
P7	Text garbling in generated frames	Video models corrupt lettering that is baked into the panel.	Clean-plate step (mask + LaMa) before animation; re-add dialogue as subtitles or bubble overlays.	High

ID	Problem	Why it is hard	Mitigation	Risk
P8	Motion semantics	A static pose does not specify motion; action panels imply movement that is not drawn.	VLM writes motion intent (camera, subject, ambient) and an intensity score; per-shot-type templates; editable at CP2.	Medium
P9	Cross-panel continuity	No true in-betweens; independently generated clips cut hard against each other.	Transition rules by scene change and camera direction; optional last-frame chaining (experimental).	Medium
P10	Resolution and aspect mismatch	Panels have arbitrary aspect ratios; video models expect fixed sizes.	Crop-safe zones or blurred-fill letterbox; generate at 480-720p, then anime upscalert.	Medium
P11	Audio design	Voice per character, emotion and timing; no lip-sync in v0.1.	Voice map in the character bible; emotion tags from the VLM; clip duration from dialogue length; subtitles as safety net.	Medium
P12	Compute, VRAM and latency	Video models need roughly 8 to 80 GB depending on variant; swapping models dominates runtime.	Compute tiers; quantized variants; one heavy model resident; batch work by model; content-hash cache.	High
P13	Evaluation without ground truth	There is no reference anime version of a panel.	Proxy metrics (identity similarity, flicker, OCR-recovery) plus a fixed human rubric on a fixed eval set.	Medium
P14	Orchestration reliability	Long GPU jobs fail midway; sampling is nondeterministic.	Checkpointed state machine; idempotent nodes; seeds and configs logged; retry then fallback.	Medium
P15	Licensing and rights	Open-weight licenses differ (Apache/MIT versus community licenses with revenue or region limits); source art is copyrighted.	License register per model; prefer Apache/MIT; keep outputs private unless rights are cleared.	Medium
P16	Model-landscape volatility	Rankings shift monthly and sources disagree on VRAM and licenses.	Provider interface per stage, config-driven model choice, repeatable bake-off harness.	Low

6. Open-source model selection

Legend: ✓ checked against a web source on 6 Oct 2026 · † general knowledge, verify on the model card. Sources often disagree on VRAM and license terms; where they do, both figures are shown and the model card is the authority.

6.1 Perception and understanding

Stage	Primary	Fallback / alternative	License (as reported)	Notes
Panel slicing	OpenCV gutter / contour slicing ✓ (as in ACMP)	Fine-tuned YOLOv8 ✓ (ACMP option)	OpenCV Apache-2.0†; Ultralytics YOLO AGPL-3.0†	Check AGPL before shipping YOLO; Magiv3 avoids it.
Panels, text, characters, tails, speaker links, OCR	Magiv3 (0.8B) ✓	Magiv2 ✓	See model card (not verified)	Manga-trained; test on manhwa first. v2 adds chapter-wide naming ✓.
Text mask / bubbles	comic-text-detector ✓	speech-bubble-segmentation ✓	Wrapper Apache-2.0 ✓; weights not verified	Mask feeds the LaMa step.
OCR	Qwen3-VL OCR (32 languages) ✓	Manga OCR (Japanese) ✓; PaddleOCR (KO/JA/EN)†	Apache-2.0 for Qwen3-VL 8B ✓	Pick OCR by detected language.
Scene understanding	Qwen3-VL-8B-Instruct ✓	Qwen3-VL-4B / 2B (small GPU) ✓; 32B (rented GPU) ✓	Apache-2.0 ✓	Native structured-output support ✓.
Depth / layers for parallax	Depth Anything V2 Small†; rembg ✓ (as in ACMP)	MiDaS ✓; SAM 2†	Depth Anything larger variants non-commercial†	Only needed for the parallax lane.

6.2 Cleanup and video generation

Role	Model	License	Reported hardware	Strengths / cautions
Text removal	LaMa ✓ (lama-manga, aot-inpainting also ✓)	LaMa Apache-2.0 ✓	Light (runs on CPU or small GPU) ✓	manga-image-translator bundles the same idea but is GPL-3.0 ✓; do not embed its code unless GPL is acceptable.
Video: primary	Wan 2.2 (TI2V-5B for dev, I2V-A14B for quality) ✓	Apache-2.0 ✓	5B: ~8-12 GB with FP8/GGUF ✓. A14B: community FP8 + offload 16-24 GB ✓, official path ≥80 GB ✓.	Clips ≈5 s ✓. One anime-focused blog test favored Wan (2.7) for keeping line work intact ✓; that is a single non-independent test.
Video: candidate	HunyuanVideo 1.5 (8.3B, released 20 Nov 2025) ✓	Tencent Community License ("other") ✓; one source reports regional exclusions ✓	~14 GB with offloading ✓	Reported as strongest on faces ✓. Read the license before any public use.
Video: candidate	LTX-2.3 (22B) ✓	Community license, free under \$10M revenue ✓	24-32 GB at 720p FP8 per one source; "from 12 GB" per another ✓	Native audio ✓; sources conflict on VRAM, so measure it.
Not eligible	Wan 2.5 (API only) ✓; Seedance, Kling, Veo (closed); MiniMax H3 (open weights, license excludes US, EU, UK, South Korea) ✓	-	-	Excluded by the open-source constraint or license.
Upscale / interpolate	Real-ESRGAN anime model†; RIFE†	Permissive†	Light†	Optional: 480p generation → 720p/1080p delivery.

6.3 Audio and platform

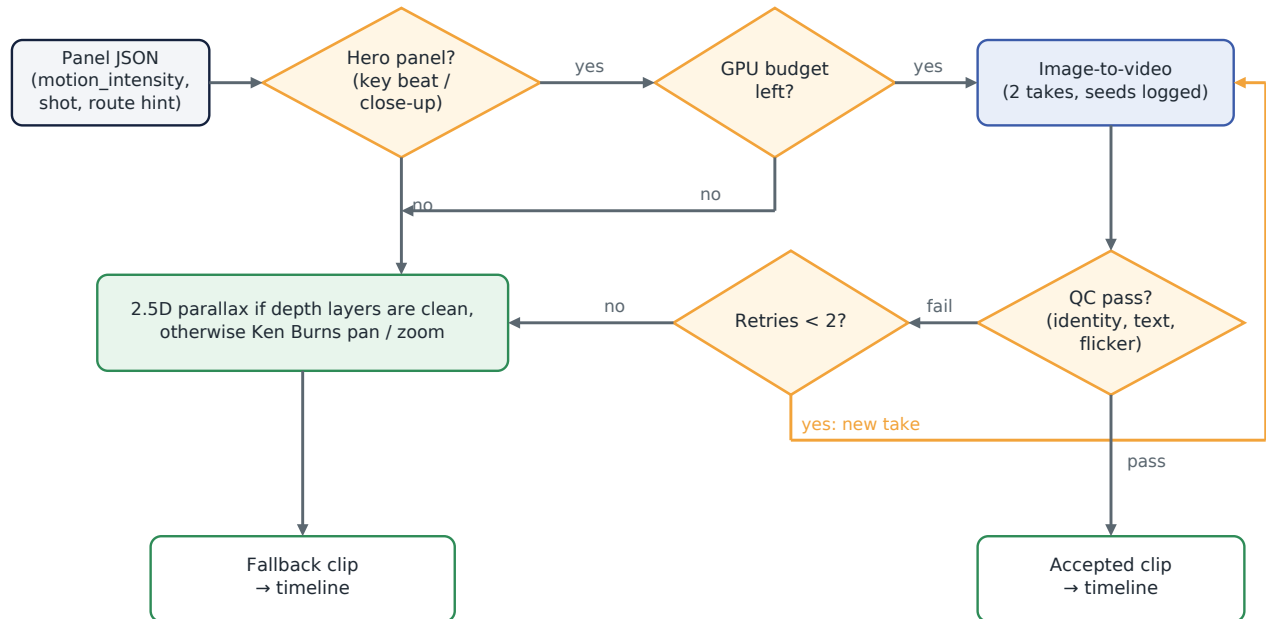
Role	Primary	Alternative	License (as reported)	Notes
Dialogue TTS	Chatterbox (~0.5B, ~4–6 GB, zero-shot cloning from ~5 s) ✓	Kokoro-82M (CPU-capable, 54 voices, 8 languages) ✓	MIT ✓ / Apache-2.0 ✓	Chatterbox for character voices; Kokoro for fast drafts.
Multilingual TTS	Qwen3-TTS (10 languages incl. Japanese, Korean) ✓	IndexTTS2 (Chinese, English) ✓	IndexTTS2 Apache-2.0 ✓; Qwen3-TTS not verified	Needed only if output must be Korean or Japanese speech.
Avoid for release	Fish Audio S2 (research license, commercial use needs paid license) ✓	F5-TTS weights (CC-BY-NC) ✓; XTTS v2 (CPML) ✓	Non-commercial ✓	Fine for private experiments only.
SFX / music (stretch)	MMAudio†	ACE-Step†	Verify†	Out of the v0.1 critical path.
Platform	LangGraph†, FastAPI†, Redis†, ComfyUI (API mode)†, FFmpeg†	-	Permissive†	Standard, permissive tooling; swap freely.

6.4 Compute tiers

Tier	Hardware	What runs	Expected experience
T0	CPU or laptop	Slicing, Ken Burns / parallax, Kokoro, small VLM (slow)	Baseline motion comic only; no AI video.
T1	12–16 GB GPU (local or free notebook GPU, to be validated in Phase 0)	Wan 2.2 Ti2V-5B (FP8/GGUF) at 480–720p, Qwen3-VL-4B/8B quantized, Chatterbox	Minutes per 5 s clip; enough for the prototype's hero panels.
T2	24–48 GB rented GPU (RTX 4090 / A6000 class)	Wan 2.2 A14B FP8, HunyuanVideo 1.5, LTX-2.3, Qwen3-VL-32B	Best quality for the bake-off and final render.
T3	80 GB datacenter GPU	All models at full quality	One source estimates ~\$0.40–0.48 per 5 s 720p Wan 2.2 clip on an H100 ✓; that is about \$5 for a dozen clips, an estimate to re-check at rental time.

7. Shot routing and quality control

The router decides, per panel, between AI image-to-video and the cheaper fallback lane. The QC gate decides whether a generated clip is allowed into the timeline.



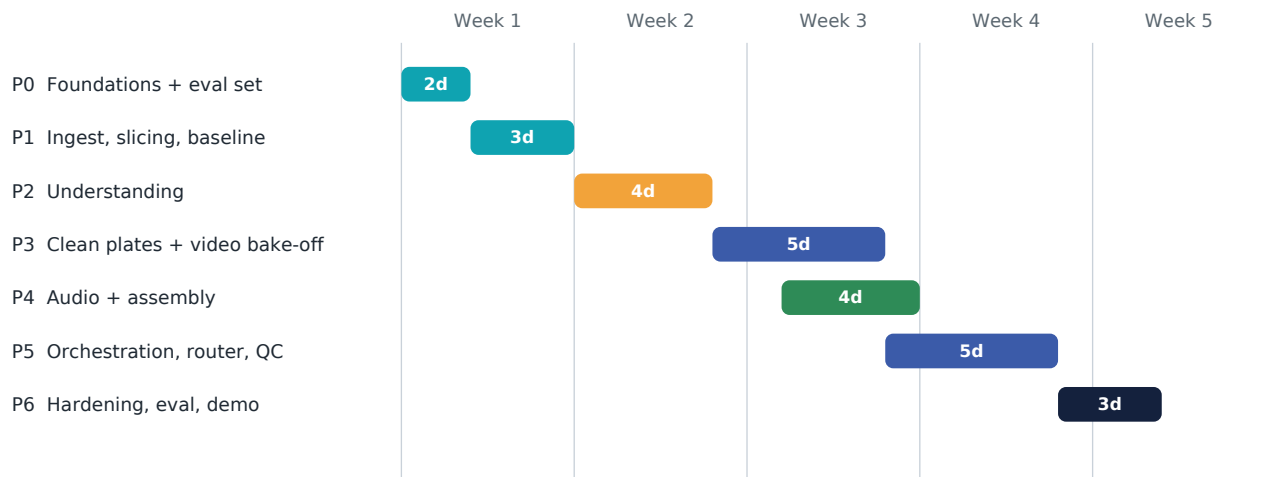
Thresholds (hero score, QC similarity) are calibrated in Phase 3 on labelled good / bad clips;
budget = maximum GPU-seconds per run; any exhausted path degrades to the fallback lane, never to a failed run.

Diagram 3. Shot router and QC loop. Every failure path ends in the fallback lane, so the run never fails because one panel failed.

QC checks (all automatic, thresholds calibrated in Phase 3)

- **Identity:** face or CLIP-embedding similarity between the source panel, first frame and last frame.
- **New text:** run OCR on sampled frames; any text that was not in the clean plate is a failure.
- **Flicker and drift:** frame-difference statistics and color-histogram drift against the source panel.
- **Motion sanity:** reject near-static clips (nothing moved) and extreme camera shake.
- **Optional judge:** ask the VLM to compare the first and last frames and answer a fixed yes/no rubric; used only as a tie-breaker.

8. Phase-by-phase implementation plan



Indicative, one developer, working days. P4 overlaps P3 (audio is independent of video). ≈22 working days total.

Diagram 4. Indicative schedule (one developer, working days).

Phase 0 · Foundations and evaluation set (≈2 days)	
Goal	Fix the yardstick before building anything.
Tasks	- Choose 3 own or licensed pages; hand-label panels, text, speakers and expected reading order. - Create repo, environment, model downloader, config system and run-folder logging. - Confirm the compute tier; start the license register (Section 5, P15).
Deliverables	- Eval set plus ground-truth JSON - Repo skeleton with stub nodes - License register v0
Exit criteria	A dry run executes every node with stubs and writes artifacts to a run folder.
Phase 1 · Ingest, slicing and baseline motion comic (≈3 days)	
Goal	Get a watchable clip early with zero generative video.
Tasks	- Stitch pages into one strip; gutter slicing; Magiv3 or detector fallback; reading-order sort. - Implement Ken Burns and 2.5D parallax renderers; FFmpeg 9:16 export. - Report panel-detection precision, recall and F1 on the eval set.
Deliverables	- Baseline MP4 (no AI video, no audio) - Panel-detection report
Exit criteria	Panel F1 ≥ 0.90 on the eval pages (or documented manual fixes) and the baseline clip plays cleanly.
Phase 2 · Understanding (≈4 days)	
Goal	Turn pixels into the panel-JSON contract.
Tasks	- Text detection and language-routed OCR; Magiv3 character clusters and speaker links. - Qwen3-VL structured JSON per panel (schema-constrained); character bible with crops, descriptors and voice assignment. - Minimal checkpoint UI (CP1) to correct JSON.
Deliverables	- Panel JSON for all eval panels - Character bible - OCR error-rate and speaker-accuracy report
Exit criteria	At least 90% of dialogue lines have correct text and speaker, or are flagged for CP1 review.

Phase 3 · Clean plates and video bake-off (≈5 days)	
Goal	Choose the video model with evidence instead of leaderboards.
Tasks	- Text mask plus LaMa clean plates; aspect-ratio handling. - ComfyUI workflows for Wan 2.2 (5B and A14B), HunyuanVideo 1.5 and LTX-2.3; motion-only prompt templates with style anchors. - Bake-off on 6 panels: rubric score, QC metrics, seconds and VRAM per clip; calibrate QC thresholds.
Deliverables	- Bake-off report - Chosen primary and fallback model config - Calibrated QC thresholds
Exit criteria	At least 70% of hero-panel clips pass QC within two takes with the chosen model.
Phase 4 · Audio and assembly (≈4 days, overlaps Phase 3)	
Goal	Add voices and a proper timeline.
Tasks	- TTS per character with emotion tags; clip durations from dialogue length. - Subtitles or bubble overlays; transition rules; audio mix and loudness normalization.
Deliverables	First full clip with audio and subtitles
Exit criteria	Dialogue is audible, matches panel timing and does not clip.
Phase 5 · Orchestration, router and QC (≈5 days)	
Goal	Make it one command and make it robust.
Tasks	- LangGraph state machine with CP1 and CP2 checkpoints; shot router; automatic QC; retry-then-fallback. - Content-hash cache; cost and time ledger; FastAPI plus Redis job API; minimal Streamlit UI.
Deliverables	- One-command run - Job API - Metrics log
Exit criteria	Resume-after-crash works; the same config and seeds reproduce routing and artifacts.
Phase 6 · Hardening, evaluation and demo (≈3 days)	
Goal	Prove it works and document what does not.
Tasks	- Regression run on the eval set; blind rubric scoring by at least three reviewers; failure taxonomy. - README, demo reel and next-step roadmap (LoRA per character, longer chapters, last-frame chaining).
Deliverables	- Evaluation report - Demo clip - Roadmap
Exit criteria	Section 9 targets are met, or each miss is explained with a proposed fix.

9. Evaluation plan and success criteria

Metric	How it is measured	Prototype target
Panel detection F1 (IoU 0.5)	Against hand-labelled panels	≥ 0.90
Dialogue text error rate	Character error rate against hand transcript	≤ 10%
Speaker attribution accuracy	Against hand-labelled speakers	≥ 85%
Identity consistency	Face or CLIP similarity: source panel vs first and last frame	Threshold set in Phase 3 from good and bad examples
New text in generated frames	OCR on sampled frames vs clean plate	0 new text lines in accepted clips
Temporal stability	Frame-difference and color-drift statistics	Threshold set in Phase 3
Human rubric (1-5)	Style fidelity, motion appeal, story clarity, audio fit; three blind reviewers	Mean ≥ 3.5, no panel below 2
Hero-clip pass rate	Share of hero panels accepted within two takes	≥ 70%

Metric	How it is measured	Prototype target
Cost and time	GPU-seconds and wall-clock per output second	Reported only (no target in v0.1)
Reproducibility	Same seed and config give the same routing and artifacts	Routing identical; video need not be bit-exact

10. Open questions and next steps

Questions that change the plan

- **GPU access:** local VRAM, or budget to rent a 24–48 GB GPU? This sets the compute tier (Section 6.4) and which video models enter the bake-off.
- **Source language:** Korean, Japanese or English? It decides OCR routing, and whether TTS must be Korean or Japanese.
- **Sample pages:** which pages, and do you hold the rights to use them?
- **Output format:** is 9:16 and 15–40 s right, and should dialogue be spoken, subtitled or both?

Immediate next steps

- Answer the four questions above and confirm the compute tier.
- Start Phase 0: select and label the eval pages, create the repo skeleton and license register.
- Run a one-hour smoke test of Magiv3 and Qwen3-VL on one manhwa page to learn early whether manga-trained perception transfers.

Honest limits of this document: the market and model facts come from public web pages and blog comparisons, several of which disagree with each other; none of the model combinations has been run on your pages yet. Phase 3 exists to replace claims with measurements.

Appendix A. Suggested repository layout

manhwa2anime/	
configs/	default.yaml, tiers/{t0,t1,t2}.yaml, models.yaml
src/m2a/	
state.py	pydantic models: PanelJSON, CharacterBible, RunState
graph.py	LangGraph definition, checkpoints, retry/fallback edges
nodes/	ingest.py segment.py understand.py clean_plates.py
	route.py generate.py qc.py tts.py assemble.py
providers/	video/{wan22,hunyuan15,ltx23}.py vlm/qwen3vl.py
	ocr/{qwen,manga_ocr,paddle}.py tts/{chatterbox,kokoro}.py
render/	kenburns.py parallax.py transitions.py
eval/	metrics.py rubric.py bakeoff.py
api/	FastAPI app, Redis worker endpoint
ui/	streamlit_app.py (checkpoint editor)
workflows/comfyui/	wan22_i2v.json hunyuan15_i2v.json ltx23_i2v.json
data/eval/	3 pages + ground-truth JSON (not committed if copyrighted)
runs/<run_id>/	panels/ clean/ clips/ audio/ state.json ledger.json out.mp4

Appendix B. Sources consulted (6 October 2026)

Gathered through web search; blog comparisons are marked as such and should be treated as claims, not measurements.

Existing systems

- <https://github.com/vc-tr/acmp>
- <https://github.com/tongli-2026/anime-pipeline>
- <https://github.com/ragavsachdeva/magi>
- <https://arxiv.org/html/2401.10224v3> (Manga Whisperer / Magi)
- <https://arxiv.org/html/2503.23344v1> (From Panels to Prose / Magiv3)
- <https://www.cometapi.com/how-to-turn-comics-into-animations-with-seedance-2-0-via-cometapi/> (blog: drift on stylized panels)

Video models

- <https://www.thundercompute.com/blog/wan-2-2-comfyui-ai-video-model> (blog)
- <https://www.hivenet.com/post/wan-2-2-cloud-gpu-comfyui> (blog)
- <https://willitrnai.com/blog/wan-2-2-vram-requirements> (blog)
- <https://www.spheron.network/blog/deploy-wan-2-5-gpu-cloud/> (blog)
- <https://www.spheron.network/blog/ai-video-generation-gpu-guide/> (blog)
- https://pinggy.io/blog/best_video_generation_ai_models/ (blog)
- <https://www.thundercompute.com/blog/best-open-source-ai-video-generation-models> (blog)
- <https://insiderllm.com/guides/local-ai-video-generation/> (blog)
- <https://github.com/Tencent-Hunyuan/HunyuanVideo-1.5/wiki>
- <https://morphed.app/blog/ai-video-generator-for-anime> (blog: anime test)

Understanding, OCR and cleanup

- <https://playground.roboflow.com/models/qwen/qwen3-vl-8b-instruct>
- <https://comfyui-wiki.com/en/models/qwen/qwen3-vl>
- <https://docs.rs/crate/koharu-models/latest>
- <https://github.com/mvanhorn/koharu>
- <https://localaimaster.com/blog/translate-manga-locally>
- <https://replicate.com/zf-kbot/manga-translator>

Audio

- <https://localaimaster.com/blog/best-local-tts-models> (blog)
- <https://github.com/ruanyl/fish-speech>
- <https://github.com/0xSojalSec/free-voice-clone> (model list)
- <https://www.spheron.network/blog/deploy-open-source-tts-gpu-cloud-2026/> (blog)